

Adapting BERT for NLP tasks

One pretrained `bert-base-uncased` body, four classical NLP tasks, and three rungs of the adaptation ladder. For every task the delivered model is the one the measurements chose, reported next to the alternatives we trained and rejected.

Team: Jose Angel Pech Xool · Valeria Nicol Hernandez Leon · Lorena Danae Perez Lopez · Julio Cesar de Aquino Castellanos · Gustavo Fuentes Marin · Ricardo Horta Sanchez

Professor: Dexter Gomez · **Course:** Trends in Data Science · **Date:** September 2026

Repository: github.com/joseangel0/adapting-bert-nlp-tasks

Deliverables: this report, the code that produced every number in it (`scripts/run_experiments.py` reruns the whole grid), and four published models with model cards. Environment: Apple M5 - torch 2.14.0 on mps · 6 runs · 33 minutes of training.

- | | |
|--------------------------------------|----------------------------|
| 1. What we delivered | 6. Results |
| 2. Task environment | 7. Training diagnostics |
| 3. Hypotheses | 8. Analysis and trade-offs |
| 4. The adaptation ladder, concretely | 9. Publishing |
| 5. Metrics and evaluation | 10. Reproducibility |
| | 11. References |

1. What we delivered

Task	Delivered model	Method	Score	Trainable
Topic classification (AG News)	<code>bert-base-uncased-agnews-topic</code>	Feature-based (logistic regression, mean-pooled)	90.18 accuracy	3,076

Each of these four models was selected against alternatives trained by us on the same data with the same seed, and every rejected alternative is reported with its numbers in section 6. The short version of what the measurements say:

The gap between a frozen BERT and a fine-tuned one is not a property of BERT — it is a property of the task. It is `{{VAL_pos_gap}}` points on part-of-speech tagging, where the information the head needs is already sitting in the pretrained representation, and `{{VAL_qa_gap}}` points on extractive QA, where it is not. Choosing the rung of the ladder without measuring is choosing blind.

2. Task environment

All four tasks share one body, `bert-base-uncased` (110 M parameters, 12 encoder layers, hidden size 768). Only the head changes: a sequence classifier over `[CLS]` for topic classification, a per-token classifier for NER and POS, and a two-logit span head (start, end) for extractive QA.

Task	Dataset	Train	Dev	Test	Labels	License
Topic classification	<code>fancyzhx/ag_news</code>	20,000	5,000	7,600	4	Custom (AG News corpus, academic use)

AG News ships no development split, so we carved one out of the training pool with the fixed seed: 20 000 rows for training, 5 000 held out to watch the epochs, and the official 7 600-row test set kept untouched for the numbers in this report. SQuAD is subsampled to 15 000 training examples as the assignment specifies; its test

split is not public, so — as is standard for SQuAD v1.1 — the 10 570-question development set is the held-out evaluation set, and a fixed 3 000-question subsample of it is used for the cheaper per-epoch monitoring.

Subword handling

The tokenizer splits words into pieces (`playing` → `play` + `##ing`) but the datasets carry one label per whole word. We follow the standard convention: **the label goes on the first subword of each word**, and every other position — continuation subwords, `[CLS]`, `[SEP]`, padding — gets `-100`, the index `CrossEntropyLoss` ignores. The same first-subword position is used when extracting frozen features, so features and labels stay aligned by construction rather than by luck.

This is the failure mode that does not announce itself: a misalignment either crashes on mismatched lengths or, far worse, trains quietly against shifted labels and produces a healthy-looking loss curve. So before any training ran, `scripts/sanity_check.py` printed tokens next to their labels and checked the invariants. Its output is committed as `docs/sanity_check.txt`; an extract:

```
=== ner: subword/label alignment (bert-base-uncased) ===
subword      word_id  label  tag
[CLS]         None    -100   (ignored)
eu             0       3    B-ORG
rejects        1       0     0
german         2       7    B-MISC
...
peter          0       1    B-PER
blackburn      1       2    I-PER

invariant check: {"checked": 500, "length_mismatches": 0,
                  "truncated_sentences": 0, "unexplained_label_loss": 0, "passed": true}
```

The machine check runs over 500 training sentences per token task and asserts that the label array is exactly as long as the input ids and that the number of labelled positions equals the number of words, unless the sentence was genuinely truncated at the maximum length. The same script decodes SQuAD's character-level answer spans back out of the subword start/end indices it produced and compares them to the gold strings, which is the equivalent check for QA: the gold answer is a character offset in the context, and it has to survive being converted into token indices inside an overlapping window that may not even contain it.

`lhoestq/conll2003` ships its NER tags as plain integers with no label names attached, so the canonical CoNLL-2003 tag order is pinned in `src/data.py` and verified at load time against the shared task's own example sentence (*EU rejects German call to boycott British lamb* ., with `EU` = `B-ORG`). A silent change in that mapping would be indistinguishable from a bad model.

Hardware

Everything was trained locally on Apple M5 - torch 2.14.0 on mps — Apple silicon, not a datacentre GPU. Two consequences shaped the design. Mixed precision through `bfloat16` autocast roughly doubles throughput on this device and is enabled for every run. And MPS has no fused attention kernel that supports dropout, so the models are loaded with the eager attention implementation; on CUDA the same code keeps the fast path. Device selection is automatic (`src/common.get_device`), so the identical modules run on Colab or Kaggle through the notebook in `notebooks/`.

3. Hypotheses

We wrote these down before running the grid, from the structure of each task and from section 5.3 of the BERT paper, where feature-based adaptation on CoNLL NER comes within about 0.3 F1 of full fine-tuning when the

features are a concatenation of the top four layers — a detail worth holding onto, because we use the last layer alone.

#	Hypothesis	Reasoning	Verdict
H1 · Topic	Feature-based adaptation gets within a few points of fine-tuning.	Topic is a document-level, largely lexical decision; a pooled sentence vector should already separate <i>Sports</i> from <i>Business</i> without reshaping the encoder.	{{VERDICT_H1}}
H2 · NER	Fine-tuning is required; the frozen gap is large.	Entity boundaries are a contextual, structured decision, and the BIO scheme means a half-recovered span scores zero. A linear read-out of the last layer has no way to reshape the representation for it.	{{VERDICT_H2}}
H3 · POS	Frozen features are strong; partial fine-tuning closes most of what is left.	Probing work consistently finds part-of-speech information in the lower and middle layers of BERT. The pretraining objective essentially has to encode it.	{{VERDICT_H3}}
H4 · QA	Feature-based adaptation fails outright.	A span head has to make the question interrogate the context. That cross-attention behaviour is not in a frozen masked-language-model representation; no linear layer on top can invent it.	{{VERDICT_H4}}
H5 · Pooling	For frozen sequence classification, mean pooling beats [CLS] .	[CLS] was pretrained for next-sentence prediction, not as a general sentence embedding; without fine-tuning there is nothing that makes it a good summary.	{{VERDICT_H5}}

4. The adaptation ladder, concretely

The three rungs are implemented by one function, `src/common.apply_adaptation`, which decides which parameters carry gradients. Everything outside the encoder body is head and always trains.

Rung	What trains	Trainable	How it is done here
Feature-based	Only the consumer of the frozen representation	0 BERT params	Two variants. (a) The last hidden states are extracted once, cached to disk, and consumed by a scikit-learn estimator (logistic regression, linear SVM, random forest). (b) A linear or MLP probe trained with PyTorch on top of the frozen body — identical in spirit, but the result is a normal <code>transformers</code> checkpoint that can be published to the Hub.
Partial fine-tuning	Head + top N encoder layers	≈ 14 M ($N=2$), ≈ 28 M ($N=4$)	The body is frozen, then the top N layers are unfrozen again. The lower layers — which probing work says carry the general lexical and syntactic machinery — are left exactly as pretraining left them.
Full fine-tuning	Everything	≈ 110 M	Every parameter carries a gradient, at the encoder learning rate.

Two learning rates, always

A freshly initialised head and a pretrained encoder want opposite step sizes: the head has no information and needs $\approx 1e-3$ to learn anything in a few epochs, while encoder weights are the product of a long pretraining run and $2e-5$ – $3e-5$ is already enough to damage them. Every run that trains head and body together therefore uses **two parameter groups** (`src/train.TwoGroupTrainer`), with a linear schedule and 10% warmup on both, and

weight decay switched off for biases and LayerNorm — the standard exception, because shrinking a normalisation gain toward zero is not regularisation, it is damage. A single shared rate would have crippled every partial and full fine-tuning run, and a crippled run inside a comparison is worse than no run at all.

The feature-based rung deserves one more note. When the body is frozen it is also **pinned in eval mode**, so its dropout never fires and it behaves as a deterministic feature extractor — otherwise the PyTorch probe and the scikit-learn estimator would be consuming different representations of the same sentence, and the comparison between them would mean nothing.

5. Metrics and evaluation

Each task is scored with the metric that penalises the mistakes that matter for it, plus a second metric that catches the specific way that task's models fail quietly.

Task	Primary metric	Why this one	Secondary
Topic classification	Accuracy	AG News is exactly balanced (1 900 test items per class), so accuracy is not misleading here and is the number the literature reports for this dataset.	Macro-F1 — identical to accuracy on a balanced set unless a class has collapsed, which is precisely the failure it is there to expose. A per-class report and a confusion matrix are stored with every run.
NER	Entity-level micro F1 (seqeval)	The unit of correctness is the whole span with its type: predicting <code>B-PER I-PER</code> as <code>B-PER 0</code> is a wrong answer, not a 50% correct one. This is the official CoNLL-2003 measure, so our numbers sit on the same scale as the published ones.	Token accuracy , kept deliberately: roughly 83% of CoNLL tokens are <code>0</code> , so a model that predicts nothing at all scores in the eighties. The distance between the two numbers is the point.
POS	Token accuracy	Every token gets exactly one tag, the tags are dense rather than sparse, and there is no span structure to respect — so the per-token decision <i>is</i> the unit of correctness. It is also the standard UD measure.	Macro-F1 over the 17 UPOS tags : accuracy is dominated by <code>NOUN</code> , <code>PUNCT</code> , <code>VERB</code> and <code>DET</code> , and hides whether the rare tags (<code>INTJ</code> , <code>SYM</code> , <code>X</code>) are learned at all. A full 17×17 confusion matrix is stored per run.
Extractive QA	F1 (token overlap, SQuAD official normalisation)	Answers are free-form spans, and a prediction that returns <i>the 1929 stock market crash</i> for a gold <i>1929</i> is largely right. F1 credits partial overlap; both are computed with the official normalisation (lowercase, strip articles and punctuation) and take the maximum over the gold answers.	Exact match — the strict complement. A large EM/F1 spread means the model finds the right region but not the right boundaries, which is a different disease from finding the wrong region.

Where the numbers come from. Every score in this report is computed on a held-out test split that was never used for training or for choosing between methods: the official AG News, CoNLL-2003 and UD English-EWT test splits, and the SQuAD v1.1 development set. The per-epoch curves in section 7 use the development splits.

Training diagnostics. Every run records its full `Trainer` log history (training loss every 50 steps, development metrics every epoch), wall-clock seconds per epoch and per step, total training time, test evaluation time, trainable and total parameter counts, and the resolved hyper-parameters — all serialised to `results/<task>/<run_id>.json`. The figures and tables in this report are generated from those files, so the prose cannot drift away from the numbers.

6. Results

The comparison the assignment asks for — tasks against methods, best run of each rung, primary metric:

Task	Metric	Feature-based	Partial fine-tuning	Full fine-tuning	Delivered
Topic classification (AG News)	Accuracy	90.18	—	—	Feature-based (logistic regression, mean-pooled)

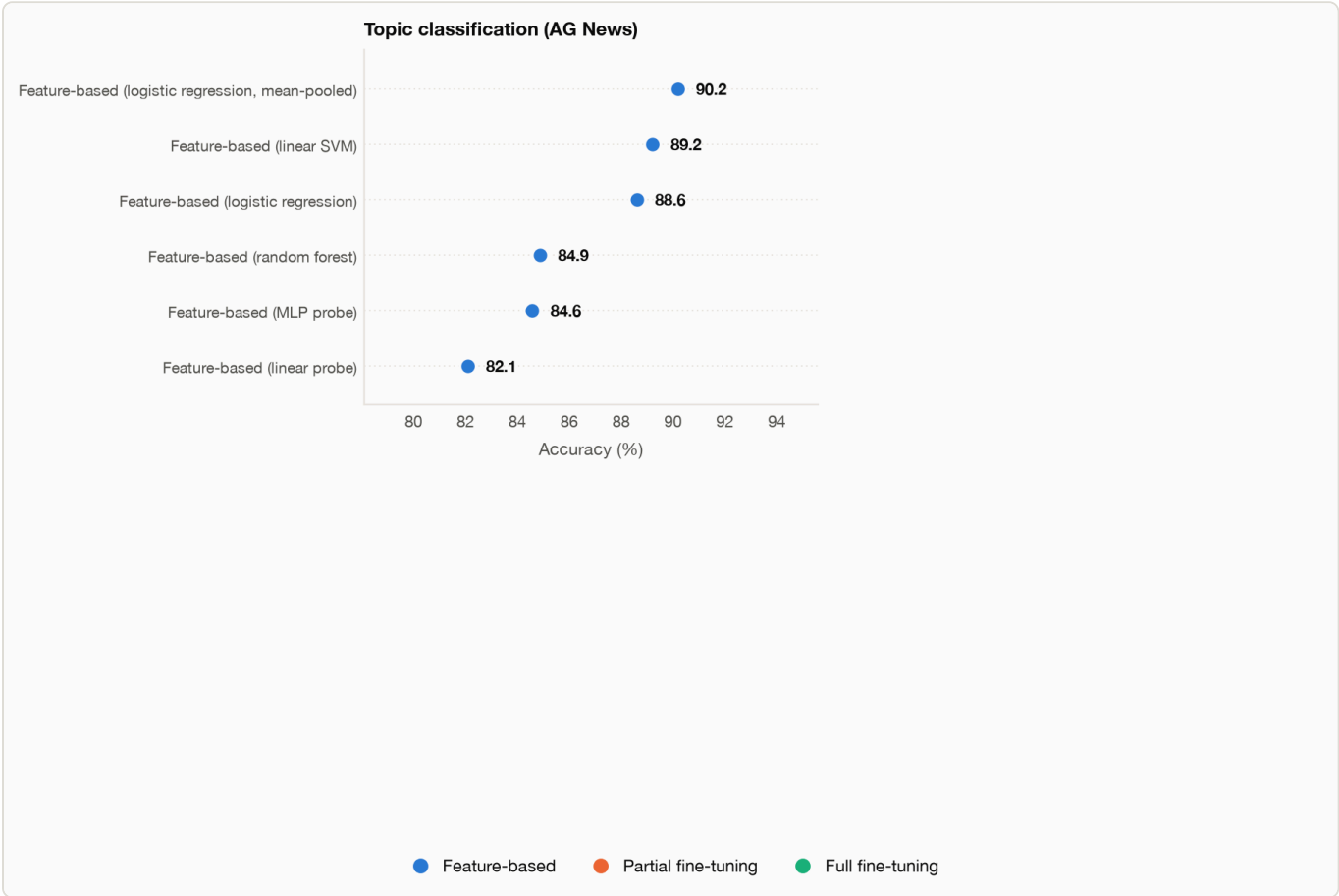


Figure 1 — every run in the grid, on its task's primary metric. Position, not bar length, encodes the score, so the compressed ranges (POS spans 90–97) stay honest without a truncated axis.



Figure 2 — what each rung costs and what it buys. The horizontal axis is trainable parameters on a log scale; the grey line joins the best run of each rung. A flat line means fine-tuning is not worth paying for.

{{SECTION_PER_TASK}}

7. Training diagnostics

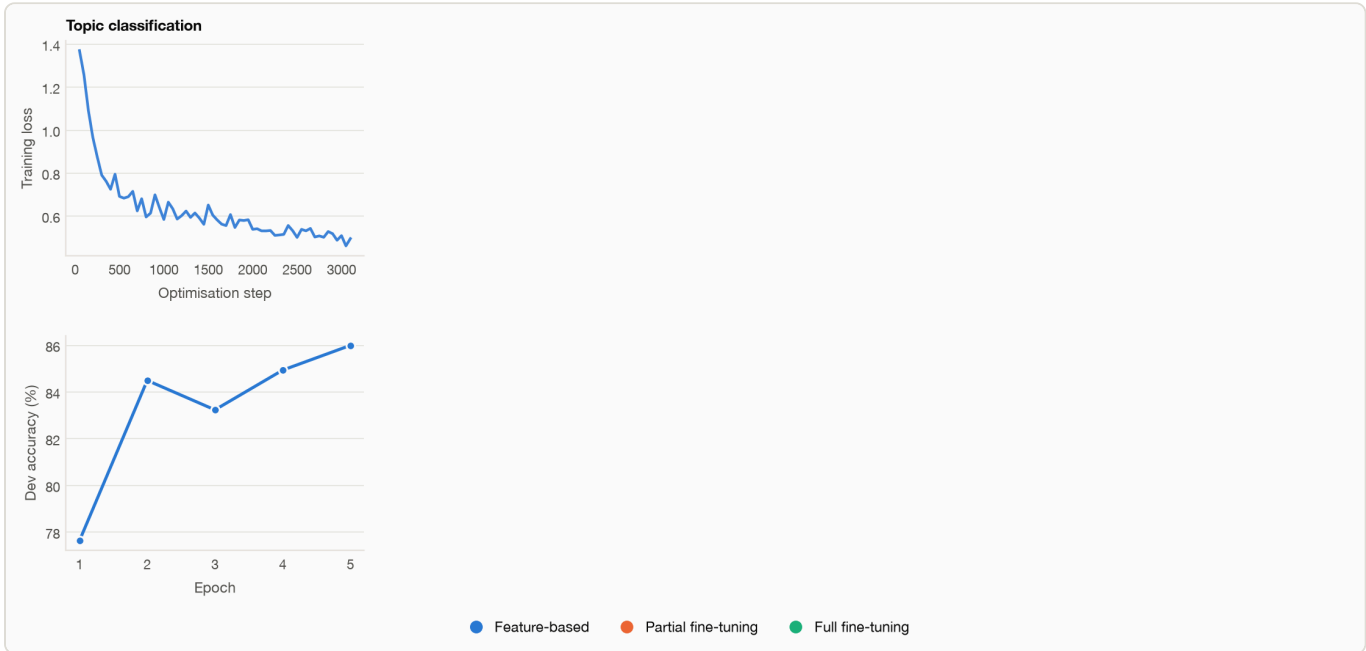


Figure 3 — top: training loss against optimisation step. Bottom: the development metric after each epoch. These are the curves used to decide that runs had converged and that no learning rate was misconfigured.

{{PROSE_DIAGNOSTICS}}

Task	Method	sec / epoch	sec / step	Total train (min)	Test eval (s)
Topic classification	Feature-based (MLP probe)	140	0.224	14.8	50
Topic classification	Feature-based (linear probe)	117	0.188	12.5	55

8. Analysis and trade-offs

{{PROSE_ANALYSIS}}

On noise. This is one run per configuration with a fixed seed, as the assignment specifies. Identical code retrained with a different seed moves results by roughly $\pm 1\text{--}3$ points — head initialisation, shuffling order and dropout all contribute — so any gap in this report smaller than that band is noise, not a finding, and we mark it as such rather than re-running it. The gaps we draw conclusions from are the large ones.

9. Publishing

The four delivered models and their tokenizers are published to the HuggingFace Hub by `scripts/push_to_hub.py`, which generates each model card from the same `results/*.json` that produced this report. Every card states the training data and its licence, the adaptation method with its full hyper-parameters, the metrics on the held-out split, **the complete table of methods we tried for that task**, intended use, limitations and bias, and references. The cards are committed in `docs/model_cards/`.

Task	Repository	pipeline_tag	Reported metric
Topic classification	<code>bert-base-uncased-agnews-topic</code>	<code>text-classification</code>	Accuracy 90.18

The repository names above are the slugs created by `scripts/push_to_hub.py --user <account>`; the account namespace is filled in from `configs/report.json` when the models are pushed.

10. Reproducibility

Seed 42 is fixed for Python, NumPy and PyTorch, and passed to `TrainingArguments` as both `seed` and `data_seed`, so head initialisation and shuffling repeat. The dataset subsamples (AG News 20 000 rows and its 5 000-row dev split, SQuAD 15 000 rows) come from seeded shuffles, so a replica sees the same rows and not merely the same number of rows. Dependencies are pinned in `requirements.txt` at the exact versions used. Reductions on GPU and MPS are not bit-exact, so the last decimal may move.

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python scripts/sanity_check.py      # subword/label alignment, before any training
.venv/bin/python scripts/run_experiments.py   # the whole grid, resumable
.venv/bin/python scripts/make_figures.py      # figures from results/*.json
.venv/bin/python scripts/build_report.py      # this report
```

The grid is resumable: a run whose result file already exists is skipped, so an interrupted session continues where it stopped. `--task`, `--limit` and `--force` narrow it while iterating, and `--limit 200` runs the entire pipeline end to end in a couple of minutes as a smoke test.

11. References

- Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL. [arXiv:1810.04805](https://arxiv.org/abs/1810.04805) — section 5.3 is the feature-based versus fine-tuning comparison this assignment mirrors.
- Tunstall, L., von Werra, L. & Wolf, T. *Natural Language Processing with Transformers*. O'Reilly, chapters 1–3.
- HuggingFace. *Fine-tune a pretrained model*, and the token-classification and question-answering task guides — the reference recipes for the `-100` convention and for SQuAD windowing and span post-processing, which `src/encoding.py` and `src/qa_utils.py` follow.
- Zhang, X., Zhao, J. & LeCun, Y. (2015). *Character-level Convolutional Networks for Text Classification*. NeurIPS — the AG News corpus.

- Tjong Kim Sang, E. F. & De Meulder, F. (2003). *Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition*. CoNLL.
- Silveira, N. et al. (2014). *A Gold Standard Dependency Corpus for English*. LREC — UD English-EWT, CC BY-SA 4.0.
- Rajpurkar, P., Zhang, J., Lopyrev, K. & Liang, P. (2016). *SQuAD: 100,000+ Questions for Machine Comprehension of Text*. EMNLP.
- Nakayama, H. (2018). *segeval: A Python framework for sequence labeling evaluation* — the entity-level scorer used for NER.
- Tenney, I., Das, D. & Pavlick, E. (2019). *BERT Rediscovered the Classical NLP Pipeline*. ACL — the layer-wise probing result behind hypothesis H3.